The P4 Ecosystem for Network Programmability

Programmable Networks A.A. 2024/25

Outline

- Abstracting a switch through a language
- P4 Language and P4Runtime Interface
- Use Cases Enabled by P4

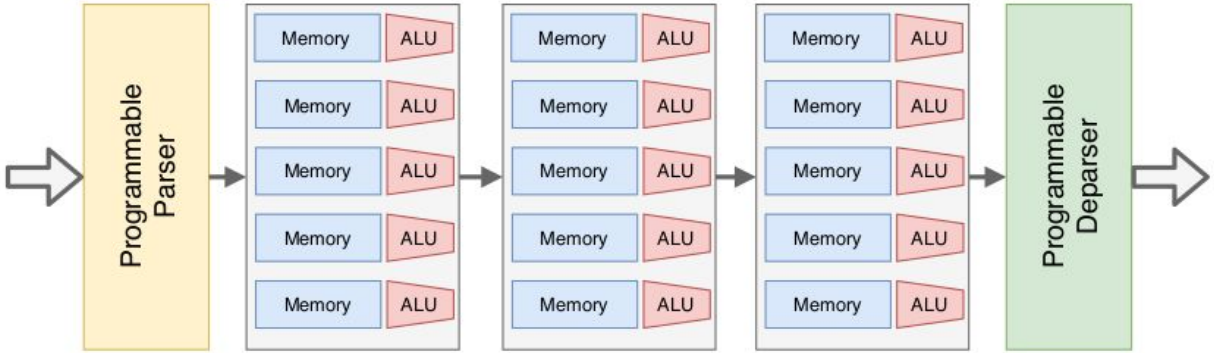
Outline

- Abstracting a switch through a language
- P4 Language and P4Runtime Interface
- Use Cases Enabled by P4

Introduction

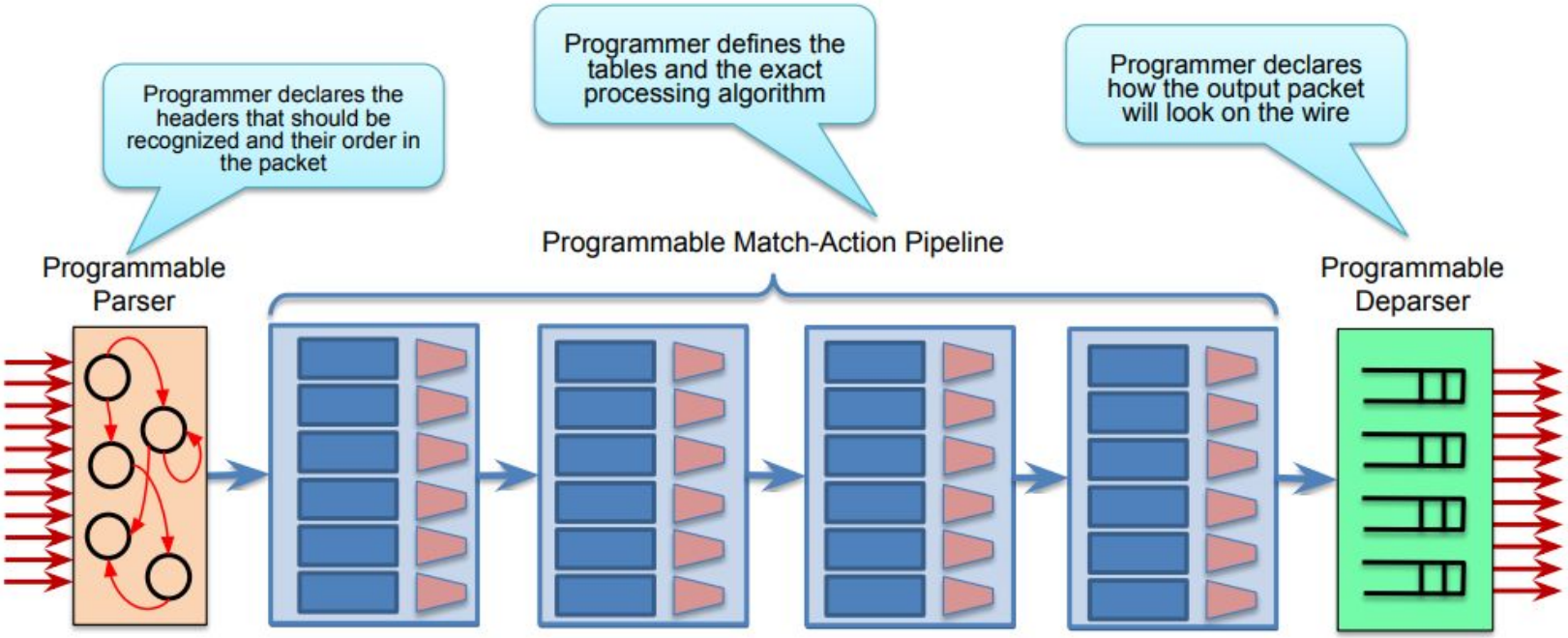
- High-speed switches and routers process multiple packets at the same time, using a multi-stage pipeline approach
- Each stage of the pipeline can be configured to operate on different combinations of header fields
  - the forwarding decision is built incrementally while packets traverse the pipeline
- The switch pipeline implementation can be composed of
  - **fixed-function stages**
    - only operate on protocols and headers known a priori and defined by the vendor
  - **programmable stages**
    - can be dynamically programmed to process user-defined header fields

PISA pipeline



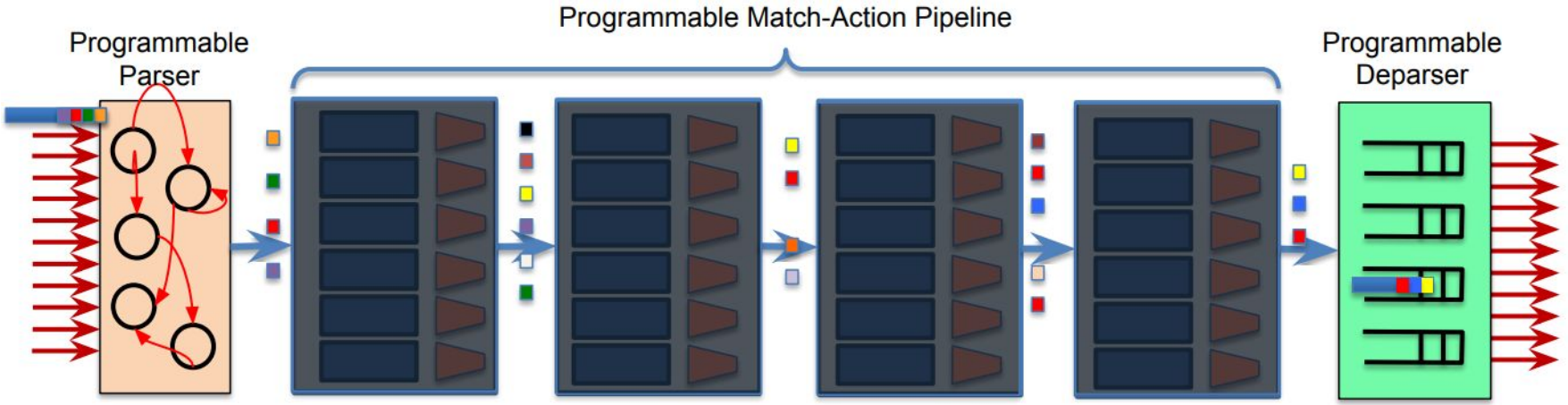
- Protocol Independent Switching Architecture (PISA) is composed of three main blocks
  - **Parser** - extract arbitrary header fields from the packet
  - **Match-Action units** - match on one or more of the extracted header fields, and to execute actions that modify the header fields or metadata
  - **Deparser** - re-serializes the headers into the packet before transmitting it on the wire
- Metadata are used to control the switch behavior and to carry information between Match-Action units

PISA: Protocol-Independent Switch Architecture



PISA in Action

- Packet is parsed into individual headers (parsed representation)
- Headers and intermediate results can be used for matching and actions
- Headers can be modified, added or removed
- Packet is deparsed (serialized)



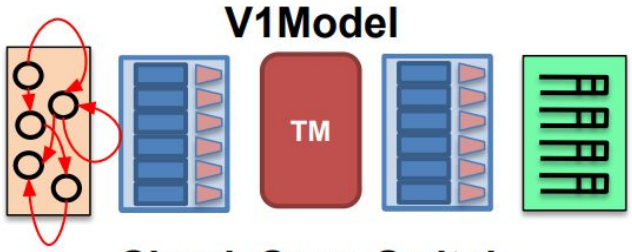
Programming Protocol-independent Packet Processors

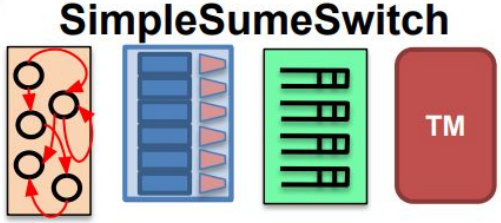
- **P4** is a high-level language for programming packet processing pipelines
- P4 is said to be target-independent, i.e., programmers
  - **can focus on** the high-level specification of how packets should be processed
  - **no need to worry** about the low-level details of the underlying hardware
- P4 Compiler
  - translates high-level representation of the pipeline onto the underlying stages
  - allocates the available hardware resources (slots in the available Match-Action units)

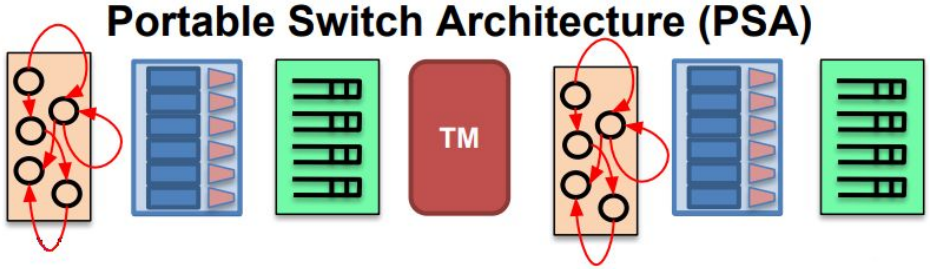
P4 Architecture

- More than ten different P4 programmable network devices are available on the market
- Heterogeneous platforms
  - different hardware resources
  - possibly different capabilities
- To account for such differences, the P4 language introduces the notion of a P4 architecture
- The architecture is defined in a sort of header file (**arch.p4**) representing a contract between the P4 program and the P4 compiler

Example Architectures and Targets







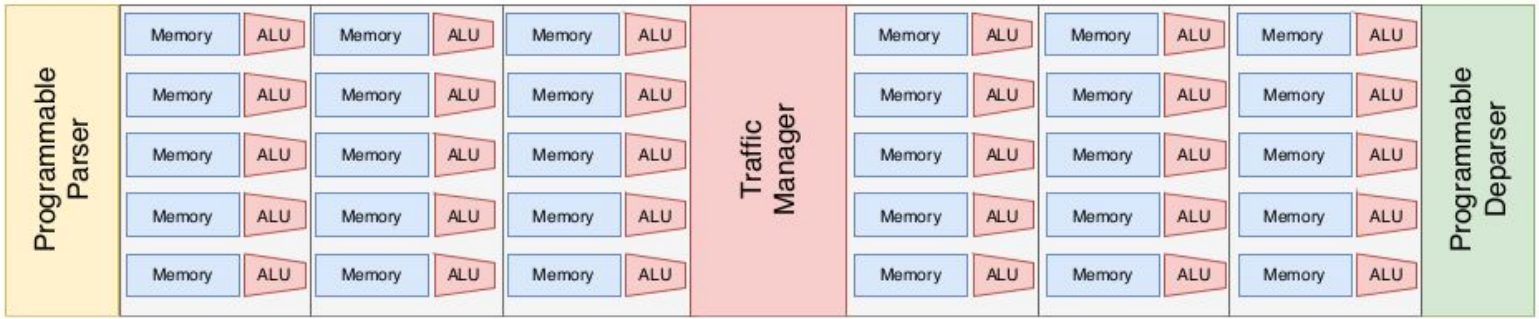
Anything

arch.P4

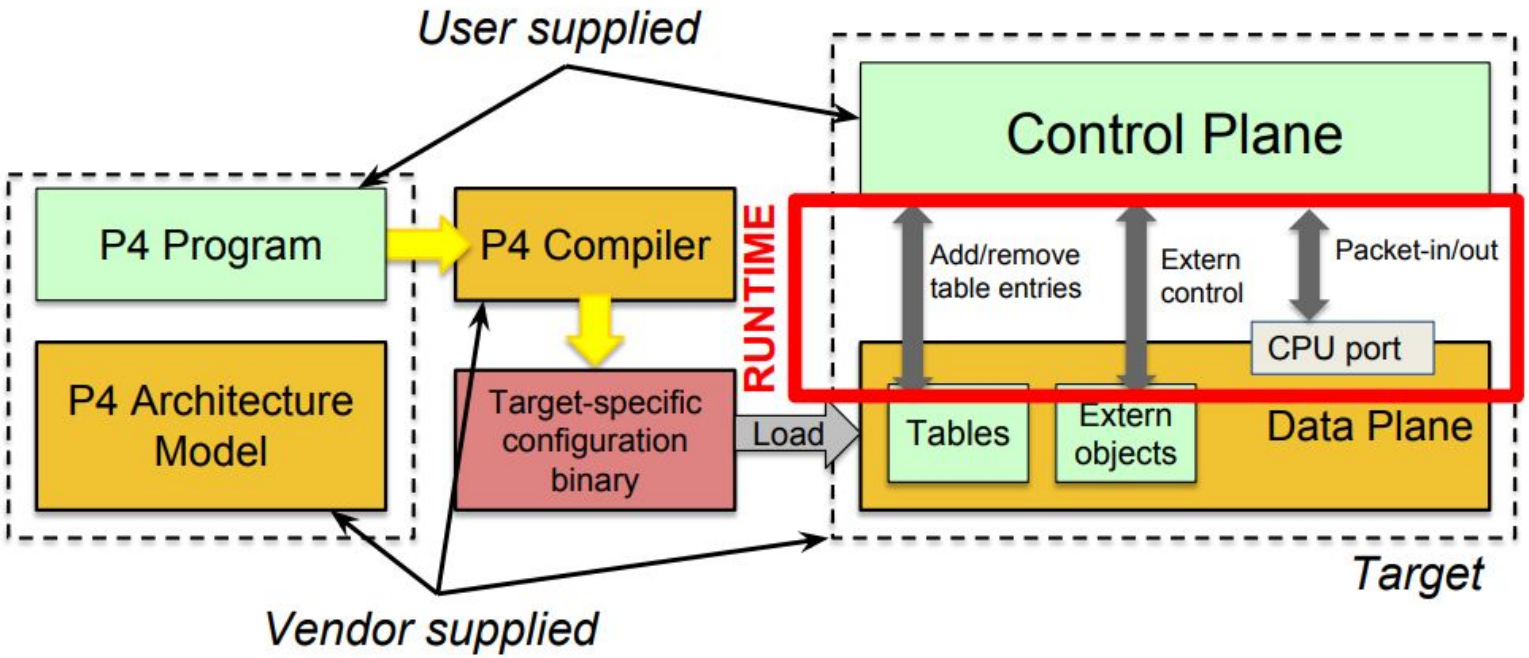
- A generic arch.p4 file defines three main elements
  - interface between the blocks of the pipeline in terms of input and output signals
  - extern functions
    - target-specific fixed-function blocks available to the P4 programmer and expose the specific features of the different targets
      - checksums, hash functions, counters, etc.
  - extensions to the core P4 language types, including alternative match types that can be available in different targets
- Example: psa.p4

V1Model Architecture

- **V1Model** introduces a fixed function element called **traffic manager**, which is responsible for queuing, replicating, and scheduling packets
- The pipeline in V1Model is divided in two halves: ingress and egress processing pipelines



Programming a P4 Target



Outline

- Abstracting a switch through a language
- P4 Language and P4Runtime Interface
- Use Cases Enabled by P4

P4 Program

- P4 language share a similar syntax to the C programming language
- Unlike C, it does not support loops, pointers, or dynamic memory allocation
- These limitations are a consequence of
  - the limited memory resources of switching ASICs
  - the purpose of P4, which is specifying the behavior of a single pipeline stage
- A P4 program must specify the following parts
  - Headers and Metadata
  - Parser
  - Ingress Processing
  - Egress Processing
  - Deparser

P4 Program Template (V1Model)

```
#include <core.p4>
#include <v1model.p4>
/* HEADERS */
struct metadata { ... }
struct headers {
  ethernet_t   ethernet;
  ipv4_t       ipv4;
}
/* PARSER */
parser MyParser(packet_in packet,
          out headers hdr,
          inout metadata meta,
          inout standard_metadata_t smeta) {
  ...
}
/* CHECKSUM VERIFICATION */
control MyVerifyChecksum(in headers hdr,
                         inout metadata meta) {
   ...
}
/* INGRESS PROCESSING */
control MyIngress(inout headers hdr,
                  inout metadata meta,
                  inout standard_metadata_t std_meta) {
  ...
}
```

```
/* EGRESS PROCESSING */
control MyEgress(inout headers hdr,
                 inout metadata meta,
                 inout standard_metadata_t std_meta) {
  ...
}
/* CHECKSUM UPDATE */
control MyComputeChecksum(inout headers hdr,
                          inout metadata meta) {
  ...
}
/* DEPARSER */
control MyDeparser(inout headers hdr,
                   inout metadata meta) {
  ...
}
/* SWITCH */
V1Switch(
  MyParser(),
  MyVerifyChecksum(),
  MyIngress(),
  MyEgress(),
  MyComputeChecksum(),
  MyDeparser()
) main;
```

basic IP forwarding program: Headers and Metadata

```
/***** HEADERS *****/
typedef bit<9>  egressSpec_t;
typedef bit<48> macAddr_t;
typedef bit<32> ip4Addr_t;
header ethernet_t {
    macAddr_t dstAddr;
    macAddr_t srcAddr;
    bit<16>   etherType;
}
header ipv4_t {
    bit<4>    version;
    bit<4>    ihl;
    bit<8>    diffserv;
    bit<16>   totalLen;
    bit<16>   identification;
    bit<3>    flags;
    bit<13>   fragOffset;
    bit<8>    ttl;
    bit<8>    protocol;
    bit<16>   hdrChecksum;
    ip4Addr_t srcAddr;
    ip4Addr_t dstAddr;
}
```

```
struct custom_metadata {
   /* empty */
}
struct headers {
    ethernet_t   ethernet;
    ipv4_t       ipv4;
}
```

basic IP forwarding program: Parser

- The programming model for the parser is a **state machine**
- The state machine
  - starts always from the built-in start state
  - terminates when either accept or reject state are reached

```
/***** PARSER *****/
parser MyParser(packet_in packet,
                out headers hdr,
                inout custom_metadata meta,
                inout standard_metadata_t standard_metadata) {
    state start {
        transition parse_ethernet;
    }
    state parse_ethernet {
        packet.extract(hdr.ethernet);
        transition select(hdr.ethernet.etherType) {
            TYPE_IPV4: parse_ipv4;
            default: accept;
        }
    }
    state parse_ipv4 {
        packet.extract(hdr.ipv4);
        transition accept;
    }
}
```

basic IP forwarding program: Ingress Processing

- In this section of the P4 Program we can define
  - actions and tables
  - the order in which they are applied

```
/***** INGRESS PROCESSING *****/
control MyIngress(inout headers hdr,
                  inout custom_metadata meta,
                  inout standard_metadata_t standard_metadata) {
    action drop() {
        mark_to_drop(standard_metadata);
    }
    action ipv4_forward(macAddr_t dstAddr, egressSpec_t port) {
        standard_metadata.egress_spec = port;
        hdr.ethernet.srcAddr = hdr.ethernet.dstAddr;
        hdr.ethernet.dstAddr = dstAddr;
        hdr.ipv4.ttl = hdr.ipv4.ttl - 1;
    }

    table ipv4_lpm {
        key = {
            hdr.ipv4.dstAddr: lpm;
        }
        actions = {
            ipv4_forward;
            drop;
            NoAction;
        }
        size = 1024;
        default_action = drop();
    }

    apply {
        if (hdr.ipv4.isValid()) {
            ipv4_lpm.apply();
        }
    }
}
```

basic IP forwarding program: Egress Processing

- This routine is useful to perform actions based on the egress port
  - e.g., if a switch is expected to send VLAN-tagged packets on a specific port

```
/***** EGRESS PROCESSING *****/
control MyEgress(inout headers hdr,
                 inout custom_metadata meta,
                 inout standard_metadata_t standard_metadata) {
    apply { /* no operations */ }
}
```

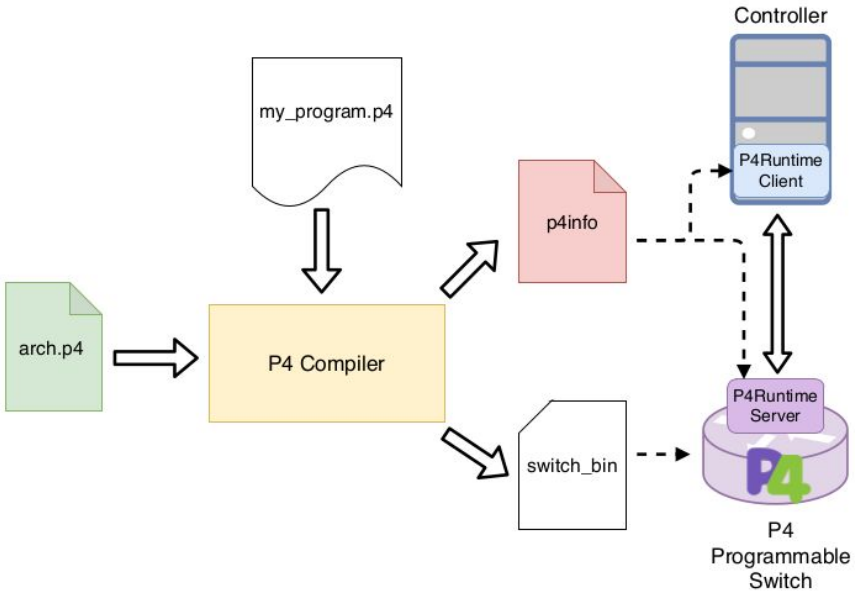
basic IP forwarding program: Deparser

- The deparser defines how the packet is re-built and prepared for being sent out from the interface
- By default, all the bytes beyond the latest parsed fields are included in the outgoing message
  - in this way there is no need to explicitly de-parse the packet payload, or the non-parsed headers

```
/***** DEPARSER *****/
control MyDeparser(
                packet_out packet,
                in headers hdr) {
    apply {
        packet.emit(hdr.ethernet);
        packet.emit(hdr.ipv4);
    }
}
```

Controlling switches at run-time with P4Runtime

- P4Runtime is the control-plane interface to control switching data planes defined in P4
- You can think of P4Runtime as the equivalent to OpenFlow for P4 programmable devices
  - OpenFlow is protocol-dependent
  - 4Runtime, instead, allows to control custom pipelines defined in P4



P4Runtime Contract

- The P4Runtime interface is composed of
  - a server-side RPC stub controlling the **switch**
  - a client-side stub, which is included in the **SDN Controller**
- Together, they implement the P4Runtime Contract between the controller and the switch
- The compiler generates a corresponding .p4info file that specifies the P4Runtime Contract

```
tables {
    preamble {
        id: 37375156
        name: "MyIngress.ipv4_lpm"
        alias: "ipv4_lpm"
    }
    match_fields {
        id: 1
        name: "hdr.ipv4.dstAddr"
        bitwidth: 32
        match_type: LPM
    }
    action_refs {
        id: 28792405
    }
    action_refs {
        id: 25652968
    }
    action_refs {
        id: 21257015
    }
    size: 1024
}
actions {
    preamble {
        id: 21257015
        name: "NoAction"
        alias: "NoAction"
    }
}
actions {
    preamble {
        id: 25652968
        name: "MyIngress.drop"
        alias: "drop"
    }
}
actions {
    preamble {
        id: 28792405
        name: "MyIngress.ipv4_forward
    "
        alias: "ipv4_forward"
    }
    params {
        id: 1
        name: "dstAddr"
        bitwidth: 48
    }
    params {
        id: 2
        name: "port"
        bitwidth: 9
    }
}
```

Outline

- Abstracting a switch through a language
- P4 Language and P4Runtime Interface
- Use Cases Enabled by P4

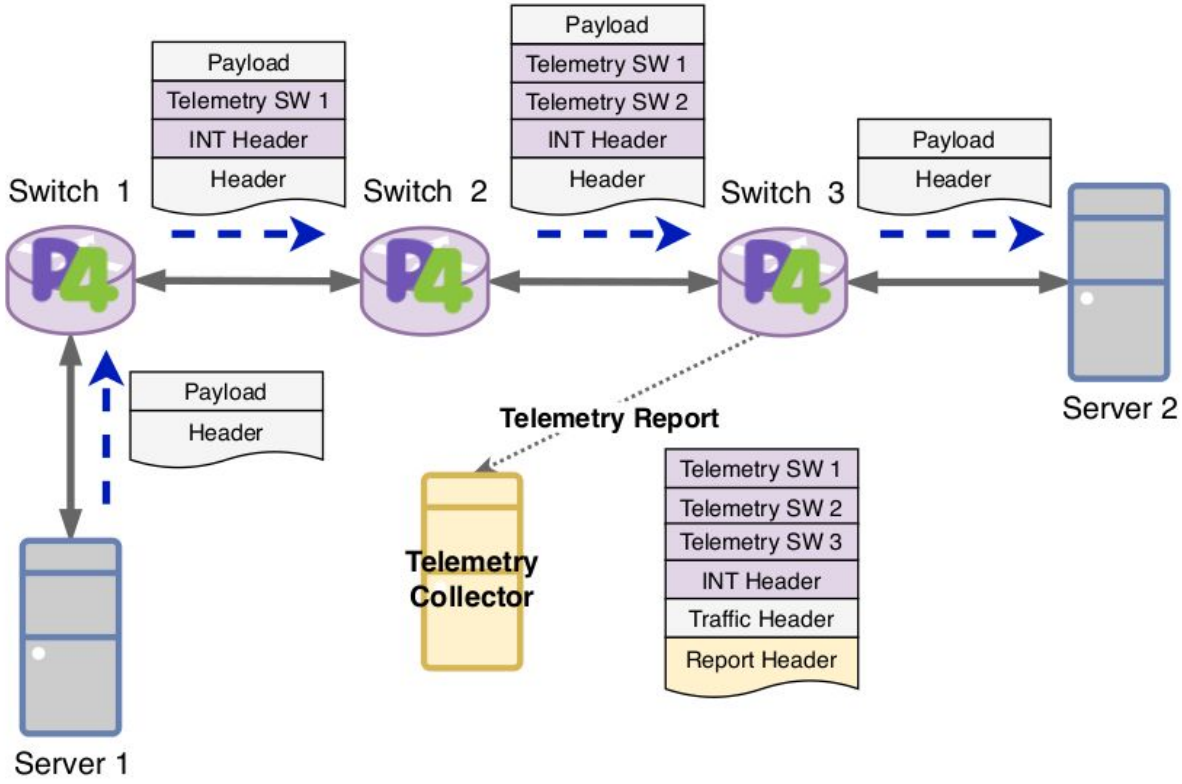
Use cases

- In the next two use cases are detailed
  - In-band Network Telemetry (INT)
  - Software Defined Broadband Network Gateway (SD-BNG)

In-band Network Telemetry

- INT allows to collect and report network statistics without having the control plane continuously polling the network devices
- INT works by injecting a custom header into packets flowing through the data plane
- These packets can be
  - normal data packets
  - packet clones
  - special probes
- The INT header
  - contains telemetry instructions that are interpreted by the INT-enabled devices that add the available telemetry metadata into the packet
  - is popped from the packet at the last INT-enabled device and sent to a telemetry collector for further elaboration

In-band Network Telemetry



Software Defined Broadband Network Gateway

- The BNG is the network element managing subscriber access in a broadband network
  - It manages sessions
  - aggregates traffic from different subscribers
  - routes it to the core network
- BNGs usually provide closed, proprietary APIs and require system integration to inter-operate with different vendors devices in a telco network
- With P4, the BNG functionalities can be implemented on merchant silicon providing open APIs (e.g., P4Runtime), reducing deployment and operational cost in the telco network

Software Defined Broadband Network Gateway